Paso 23 — Estructura (ablación e inversión): qué se fabrica, qué se calcula, con qué estadística y qué criterio hay cargado

Fecha: 2026-10-08 · **Para:** el dueño, que va a fijar los criterios él mismo · **Alcance:** sólo el paso 23: la fábrica `sqx/structural/` (que usa de `sqx/variants/` el sellado del fichero y la ejecución en el custodio), el estudio `studies/readings/structure/`, su capítulo del manual (`51-estructura.md`), su relación con el trader al azar (`studies/readings/monkey/`) y lo que el autopiloto hace con el paso.

Revisado por dos miembros del Council (Feynman, Taleb) el 2026-10-08; correcciones aplicadas.

Qué es: el documento de consulta del paso 23, con la misma forma que el de los pasos 15-16 (`criterios-paso-16-spp-2026-10-07`). Explica desde cero qué es una ablación, qué es el «recorte al azar» y qué demuestra (y qué no) la inversión; repasa cada número que escribe el estudio con su `fichero:línea`; cita el criterio cargado (ninguno, a propósito) y cierra con lo que no cuadra y con las decisiones que te quedan.

Commit descrito: `b2b4c8c`. **Cambios sin confirmar en los ficheros del paso (las líneas citadas son las del árbol de trabajo, no las de `b2b4c8c`):** `studies/readings/structure/` y `engines/nulls/filter.py` están limpios; `ledger/gate.py` (añade `seal` y desplaza `allow`) y `OPEN.md` también tienen cambios sin confirmar; `sqx/structural/keep.py` tiene un cambio sin confirmar (deja pasar a la sesión que tiene el custodio vivo, no cambia ningún número); `sqx/variants/execute.py` y `sqx/variants/build/rewrite.py` (lo que la fábrica reutiliza) y varios de `pipeline/autopilot/` (`facts.py`, `plan.py`, `tailrun.py`) también tienen cambios sin confirmar. `pipeline/autopilot/closing.py`, que es quien lanza el paso, está limpio.

De dónde salen los números. Los ejemplos son de la corrida `Test_USDJPY_M30_doubleROCLong`, lote `20261006-201740` (USDJPY, M30, largas, plantilla «doble ROC» + un hueco aleatorio; tres madres). Es la corrida más reciente con ablaciones, y es posterior al último cambio del estudio (2026-10-02), así que lo que guardó es lo que el código hace hoy. Todo está **recalculado por mí desde las operaciones exportadas** (`AlgoData/raw/…/WFC_Build|WFC_OOS1/2026-10-06/structure/trades.parquet`) con las funciones del propio estudio (`measure.py`, `reading.py`) y comparado con los JSON guardados: coinciden etiqueta por etiqueta y cifra por cifra. Para las cuentas de población uso además los `verdict.csv` de los otros nueve lotes que hay en `AlgoData/structural/`. Lo que el estudio no calcula —la deriva del activo, la cola contraria del p, la calibración del nulo, la potencia— lo he calculado yo y va marcado **«calculado por mí»**, con su script en la carpeta temporal `…/scratchpad/paso23/` (`recalc.py`, `mine.py`, `unablated.py`, `toy.py`). **No he abierto ningún dato de `oos2`.**

Ninguna línea de este documento está congelada. `pipeline/autopilot/criteria.yaml:59` lo dice del fichero entero, y de este paso en particular: «Steps 21, 23, 24 annotate only: no rule on purpose».

---

0. En una página

La pregunta del paso. De una estrategia que ya ha sobrevivido: ¿cuál de sus condiciones de entrada es la que elige buenas operaciones, cuál está de adorno, y gana por la dirección en que entra o por otra cosa (costes, swap, la subida del activo)?

Lo que hay cargado hoy: ninguna regla. El paso no tiene entrada en `criteria.yaml`, no tiene umbrales en `ledger/thresholds.yaml` y el autopiloto no lo juzga. Lo único que «decide» son cinco mandos del estudio (`config.yaml`): 10.000 recortes al azar, semilla 0, α = 0,05, tolerancia de identidad 0,5 % y emparejamiento mínimo 99 %. Deciden **palabras**, no supervivientes.

Lo que sale en la corrida real (tres madres, dos condiciones cada una):

	`build` (2008-2017)	**`oos1` (2018-2022)**
condiciones «aporta»	5 de 6	**0 de 6**
«no se distingue»	1	5
«resta»	0	1
inversión «en la dirección»	3 de 3	2 de 3

Y en los cinco lotes con ablaciones que hay en disco (15 madres, 30 condiciones): en `build` «aporta» **27 de 30**; en `oos1`, **1 de 30**. Con α = 0,05 el puro azar daría 1,5 de 30 como mínimo (hasta 3,6 con el nulo estrecho del §2.5; calculado por el revisor). Esas 30 condiciones son de 15 madres de cuatro proyectos de plantilla (12 de USDJPY, 3 de USA500); de las corridas sin plantilla no hay ninguna condición medida, porque la fábrica no las ablaciona (punto 1 de abajo). Y ese 1 de 30 no quiere decir que las otras 29 no aporten: quiere decir que el paso, con estas muestras, no llega a verlo (§2.8).

Las doce cosas que conviene saber antes de leer el resto:

1. **En las corridas sin plantilla (Simple Strategy) el paso no ablaciona nada.** La fábrica sólo reconoce la señal de entrada cuando el fichero la escribe como `<Item key="AND">` a secas con hijos `<Block>` (`sqx/structural/logic.py:10-11`). El generador de SQX, cuando el AND lo sortea él, escribe `<Item key="AND" generated="random" …>` con hijos `<Item>`. La fábrica lo toma por «una sola condición llamada AND» y no fabrica ablaciones. En los cinco lotes de NIKKEI225 hay **18 madres de 50 con dos condiciones de entrada y cero ablaciones** (calculado por mí). El informe no avisa: sale sólo la inversión. El manual dice que eso pasa «cuando la señal tiene una sola condición»; aquí tienen dos.
2. **El control «la madre reconstruida es la madre» no puede pasar en el autopiloto.** Compara el tramo `build` con la muestra que el fichero de la madre guarda con el código «IS» (`measure.py:12`). En el fichero que el autopiloto copia, esa muestra es el backtest de `oos1`: la madre sale de la databank `SPP OOS`, cuya tarea corre sobre `oos1`, y un retest escribe su resultado en el bloque «IS» (§3.4). Ejemplo: guardado 402 operaciones / −24.602,68; reconstruida en `build` 736 / +76.734,87 → «igual: False». La reconstrucción **sí** reproduce al céntimo el resultado guardado de `oos1` (402 / −24.602,68); la de `build` no se puede contrastar con este fichero, que no guarda nada de `build`. Es un fallo de control, no del resultado: en los diez lotes, 59 de las 62 madres con fila en `build` dan «igual: False» y en las 59 lo guardado coincide con `oos1` (calculado por el revisor, sobre los JSON guardados; las tres restantes, de un lote de 2026-09-27 que no vino del autopiloto, dan «igual: True»). El veredicto de controles sale siempre «watch» en el autopiloto. `OPEN.md:582` anota un caso emparentado (tabla vacía con madres de `CrossTF_Mothers`); el de `SPP OOS` (tabla llena y mal apuntada) no está anotado.
3. **«Aporta» en `build` no es evidencia.** El generador eligió la condición del hueco, y la estrategia entera, precisamente porque mejoraban el `build`. Que el recorte al azar lo confirme (27 de 30) es ver el molde de la selección. El dato que informa es `oos1`, y allí casi nada se distingue; pero «no se

distingue» no es «no aporta» (punto 9). La ventana y los hechos, sin embargo, abren por defecto en `build` (primer tramo del selector).

4. **El recorte al azar sólo es una prueba exacta si la madre opera un subconjunto de lo que opera la ablación.** No lo es: una estrategia con una sola posición abierta, al quitarle un filtro, entra antes y ya está dentro cuando llegaría la entrada de la madre. Las «entradas gemelas» van aquí del 20 % al 77 % (el README cita un 2 % de otra estrategia). Cuando las listas no están anidadas, el nulo es demasiado estrecho: con un 5 % nominal, la tasa de falsos «aporta» sube hasta un 15-20 % en el peor caso, que es cuando la condición quita pocas operaciones (calculado por mí, §2.5). Ese 15-20 % es la tasa si la madre no compartiera nada con la ablación; no es una cota demostrada.
5. **«Resta» y «en la dirección» no llevan ninguna prueba.** «Resta» es cualquier ΔE negativo, por pequeño que sea. «En la dirección» es «la madre gana a precio medio», con el corte en cero: una madre con −56 USD a precio medio sobre 402 operaciones es «sin filo» y otra con +19.696 ± 18.500 es «en la dirección». En `build` sale «en la dirección» en 65 madres de 65: una palabra que sale siempre no informa.
6. **Que el invertido pierda lo que la madre gana no demuestra nada: es aritmética.** Sin stops, el invertido es el espejo exacto de la madre a precio medio (emparejadas 100 %). Lo que informa de la inversión es el **desglose**: cuánto del neto es movimiento a precio medio, cuánto spread y cuánto swap. La **deriva del activo no se separa**: en `oos1` (USDJPY subió de 112,6 a 131,1) supone unos 8.000 USD de los 19.700-26.600 que las madres ganan a precio medio (calculado por mí, §3.3).
7. **El lote se reprueba también en `oos2`, aunque el estudio no lo lea.** `sqx.variants.execute` corre siempre las tres patas; quedan en disco el panel `retest_oos2.csv`, los ficheros reteseados de `WFC_OOS2` y sus filas en `retained.parquet`. El estudio se niega a leer `oos2` sólo si corre como agente autónomo —y hoy nadie lo activa—, y esa puerta no apunta nada en el ledger: del paso 23 no queda ninguna fila.
8. **Los hechos que saldrían no sirven para escribir una regla.** Son 68 por madre, sólo del tramo `build`, con el nombre del bloque dentro de la clave (`…cblock_bbwrpercent.p_e`), así que no hay una clave común a todas las estrategias; y las etiquetas («aporta», «resta») son texto y se descartan.
9. **El «1 de 30» en `oos1` no separa «no aporta» de «aporta un tercio de lo visto en `build`».** El ΔE es positivo en 24 de las 29 condiciones no redundantes (con condiciones sin efecto se esperaba la mitad) y la potencia contra un efecto de la mitad o de un tercio del de `build` es baja (21-50 %; calculado por el revisor, §2.8): el paso es ciego en las dos direcciones, y la frase «ninguna condición elige mejor que un dado» no se sostiene.
10. **La madre guarda `oos1` como «IS» porque sale de la databank `SPP OOS`.** Arreglar esa casilla (§5.2) sólo pondría en verde el control del tramo `oos1`: el fichero no guarda ningún resultado de `build`, así que ese control seguirá en «none» y la pestaña de `build` en «watch».
11. **11 ejecuciones han reprobado en `oos2` 105 versiones nuevas de estrategia sin ninguna fila en el ledger** (30 ablaciones y 75 inversiones; calculado por el revisor, sólo por existencia de ficheros). La tabla del cierre ponía «coste: 0» y no recogía este gasto del tramo reservado (§5.13).
12. **En la inversión, la diferencia entre lo que pierde el invertido y lo que gana la madre es 2·spread más los swaps y comisiones de las dos patas, y «peor que estar largo a ciegas» no se distingue de cero.** A en `oos1`, descontada la deriva, queda en −7.946 con un error típico de ±19.490 (t = −0,41): indistinguible de estar largo a ciegas (§3.3).

---

1. Qué es el paso 23 y cómo decide

1.1 Dónde está

Familia «lecturas» (`studies/readings/`), en el cierre del workflow: después del mapa condicional (22) y antes del stop ATR (24). El orden con el 24 no es casual: la inversión sólo es un espejo cuando la estrategia no lleva stop, target ni trailing, y el 24 es el que los pone.

Es un paso **por estrategia**: no hay población, no hay `many.py` , no hay ranking. Cada madre recibe su lectura.

1.2 La cadena, en cinco comandos

`pipeline/autopilot/closing.py:72-98` los lanza en este orden:

#	comando	qué hace	toca SQX
1	`sqx.structural.make`	fabrica los ficheros del lote y los relee	no
2	`sqx.variants.execute`	los carga en el proyecto, los reprueba en las tres patas WFC y para el custodio	**sí**
3	`sqx.structural.keep`	copia fuera los ficheros que SQX devolvió y comprueba que conservan la edición	no (lee ficheros)
4	`sqx.export.export_retest --batch structure`	exporta las operaciones de `WFC_Build` y `WFC_OOS1`	**sí**
5	`studies.readings.structure.report`	la lectura: una página por madre y `verdict.csv`	no

Antes del 1 el autopiloto copia el `.sqx` de cada madre a `AlgoData/structural/<proyecto>/mothers/` (vaciando la carpeta), pregunta a la puerta del ledger por `build` y `oos1` , y vuelve a dejar activas sólo las tres tareas WFC (`closing.py:84-90`). Después del 5 vacía los databanks (`closing.py:97`).

Qué costó en la corrida real: 143 segundos para tres madres (12 ficheros × 3 patas × 17 mercados, porque las patas WFC llevan los mercados adicionales; el estudio sólo lee el principal, `USDJPY_M1`).

1.3 Qué fabrica

Por cada madre (`sqx/structural/plan.py:42-54`):

fichero	qué es	para qué
`S00O00`	la madre **reconstruida**: el mismo fichero pasado por la fábrica sin tocar las reglas	la referencia: todo se compara contra ella, no contra el backtest antiguo
`S00A01` , `S00A02` …	una **ablación** por condición de entrada: la madre con esa condición borrada	qué aporta cada condición
`S00I00`	la **inversión**: mismas señales, orden al revés	dónde vive el resultado

`S00` es la madre (00, 01, 02…), la letra el tipo (O identidad, A ablación, I inversión).

Cómo se hace una ablación (`logic.py:89-115`). Es una sustitución de texto en `strategy_Portfolio.xml`: se localiza la señal `LongEntrySignal` (o `ShortEntrySignal`), se buscan los hijos directos de su AND/OR y se borra uno entero. En un AND, un término menos es «esa condición ya no se exige». No se añade ningún bloque «siempre verdadero». Los parámetros del bloque borrado quedan declarados y huérfanos; SQX carga el fichero igual.

Cómo se hace la inversión (`logic.py:134-164`). Se cambia el signo de cada `#Direction#` dentro de `<Rules>` y se permutan las palabras que nombran un lado («Long entry» ↔ «Short entry», `MarketPositionIsLong` ↔ `Short`). Las señales y las condiciones no se tocan: la estrategia entra **en las mismas barras** y sale en las mismas, pero vendida en vez de comprada.

Qué se sella. El identificador, los dos campos de nombre y la retirada de la huella heredada los pone `sqx.variants.build.rewrite.variant` (`make.py:44`): lo mismo que usa la fábrica de variantes del paso 16.5.

1.4 Las cuatro negativas de la fábrica

negativa	dónde	qué pasa
señal con una sola condición	`plan.py:46`	no se fabrica ablación (sin ella no quedaría regla de entrada); salen identidad e inversión
la madre lleva stop, target, trailing o break-even	`logic.py:153-156`	`ValueError`: no hay inversión. **Tumba el lote entero**, no sólo esa madre
el fichero escrito no es el plan	`make.py:104-110`	se relee cada `.sqx` del disco; si falta un fichero, sobrevive el bloque borrado o no cambió la dirección, `SystemExit`
SQX devolvió un fichero distinto	`keep.py:65-67`	tras el retest se relee lo que SQX guardó; si perdió la edición, `SystemExit`

La última existe por un fallo conocido (`OPEN.md` §9): SQX puede ignorar en silencio una edición y devolver el backtest de la madre. Desde el panel eso es idéntico a «la condición era redundante». `keep` es lo que distingue los dos casos.

1.5 Qué lee el estudio y sobre qué tramos

Las **operaciones** (no los resúmenes) de cada fichero, en dos tramos:

- `build` — la muestra con la que el generador construyó la estrategia.
- `oos1` — el primer tramo fuera de muestra.

`oos2` no se lee: `closing.py:16` fija `STRUCTURE = ("build", "oos1")` y `WORKFLOW.md:145` lo declara. El tramo de cada databank sale del `ran.json` del propio lote (`inputs.py:80-83`), no de la configuración de hoy.

1.6 No hay estados ni corte

En los demás pasos hay PASA / LIMBO / NO PASA y un databank del que se borra. Aquí no: el resultado de cada madre es `verdict: info` (`report.py:40`), el contrato se devuelve «sin veredicto global» (`one.py:70`) y todas las madres siguen al paso 24. El aviso fijo de cada página lo dice: «Diagnóstico, nunca selección» (`one.py:84-87`).

1.7 Las estrategias que nos acompañan

	A	B	C
nombre (`Strategy …`)	`10.23.380`	`3.13.366`	`4.25.395`
condición fija de la plantilla	doble ROC	doble ROC	doble ROC
condición del hueco aleatorio	`CBlock_BBWRPercent`	`CBlock_BBWRPercent`	`CBlock_WassersteinShiftHigh`
operaciones `build` / `oos1`	736 / 402	602 / 331	420 / 240
neto `build` (USD)	+76.735	+74.556	+62.780
neto `oos1` (USD)	**−24.603**	+294	+16.261

El «doble ROC» son dos condiciones ROC sobre dos periodos metidas en un AND propio. Para la regla de «como mucho dos condiciones» cuenta como una (regla 14), y la fábrica lo trata igual: lo borra entero, como un solo hijo. En las tablas del estudio aparece con el nombre **«AND»**, que es la clave de su elemento exterior (`logic.py:85`); aquí lo llamo «doble ROC».

A es el caso malo (pierde en `oos1`), B el plano y C el bueno. Las tres son largas.

---

2. Fundamentos de este paso

2.1 Qué es una ablación y qué cambia al hacerla

Una estrategia entra cuando se cumplen **a la vez** sus condiciones: «el ROC está alto **y** el ancho de las bandas está en tal percentil». Una **ablación** es la misma estrategia con una de esas condiciones quitada: «el ROC está alto», y nada más. Se vuelve a probar sobre los mismos datos y se compara con la original.

Al quitar una condición pasan tres cosas, y la tercera es la que complica todo:

1. **Opera más.** Un filtro menos deja pasar más entradas. En A, sin `BBWRPercent`, de 736 operaciones se pasa a 4.144.
2. **Las operaciones nuevas suelen ser peores** (si el filtro servía) **o iguales** (si no servía).
3. **Las operaciones antiguas ya no son las mismas.** La estrategia lleva una sola posición a la vez. Sin el filtro entra antes; cuando llega la barra en la que la madre habría entrado, la ablación ya está dentro de una operación abierta horas atrás, y no abre otra. Así que la lista de la ablación **no es** «la de la madre más unas cuantas»: es otra lista, que comparte con la de la madre sólo una parte de las entradas.

Esa parte compartida es lo que el estudio llama **entradas gemelas**: de las entradas de la madre, qué fracción aparece también, a la misma hora, en la ablación (`measure.py:82`). En A sin `BBWRPercent` son el 24 %; en A sin el doble ROC, el 75 %.

2.2 Por qué se compara por operación y no en beneficio total

Ejemplo de juguete. La estrategia **sin** el filtro hace diez operaciones:

```
+30  −20  +50  −10  +40  −30  +20  −40  +60  −10      suma = 90     media = 9
```

Con el filtro se queda con cuatro:

```
+30  −20  +50  +20                                   suma = 80     media = 20
```

En beneficio total gana la que no filtra (90 contra 80). Pero ha necesitado diez operaciones —diez veces el spread, diez veces el riesgo— para sacar poco más. Por operación, la filtrada gana 20 y la otra 9. La comparación en total siempre favorece a la que opera más; por eso el estudio no la usa (`engines/nulls/filter.py:7-8`).

Ejemplo real: C en `oos1`. La madre gana 16.261 USD en 240 operaciones; sin el doble ROC gana **17.545** en 348. En total «mejora» al quitar la condición. Por operación, la madre gana 67,75 y la ablación 50,42.

El estudio usa dos medidas por operación (`filter.py:9-10`):

- **Esperanza** (`expectancy`): la media del resultado por operación, en USD. En el juguete, 20 y 9.
- **Sharpe por operación**: la media dividida por la desviación típica de las operaciones. Mide lo mismo pero penalizando la irregularidad. En el juguete: la filtrada, 20 / 25,5 = 0,78; la otra, 9 / 33,6 = 0,27.

Y llama **ΔE** a «esperanza de la madre menos esperanza de la ablación»: lo que la condición añade por operación. En el juguete, ΔE = 20 − 9 = **+11**.

2.3 El recorte al azar: por qué es la vara y cómo se construye

ΔE = +11 parece bueno. Pero cualquier forma de quedarse con cuatro operaciones de diez da una media distinta de 9, unas veces mayor y otras menor, **sólo por cuáles te tocan**. La pregunta correcta no es «¿la filtrada tiene mejor media?», sino:

> si en vez de este filtro hubiera tirado un dado para decidir con qué cuatro operaciones me quedo, ¿cuántas veces me habría salido una media así de buena?

Eso es el **recorte al azar**. Se construye así:

1. Se cogen las operaciones de la estrategia **sin** la condición (las diez).
2. Se sacan al azar, sin repetir, tantas como tiene la madre (cuatro).
3. Se calcula su media.
4. Se repite muchas veces y se mira en qué fracción de las veces la media al azar es **igual o mejor** que la de la madre. Esa fracción es el **p**.

En el juguete se pueden contar todos los casos: hay 210 maneras de elegir cuatro de diez, y **52** de ellas suman 80 o más. p = 52 / 210 = **0,25**. Uno de cada cuatro recortes a ciegas iguala o mejora al filtro. El filtro, con estos datos, **no se distingue** de recortar al azar, por mucho que ΔE sea +11.

Por qué es la vara correcta y no otra:

- **Fija el número de operaciones.** Compara cuatro con cuatro, no cuatro con diez.

- **Fija el material.** Los recortes salen de las operaciones de esta misma estrategia en este mismo mercado y periodo, con sus mismos costes. La deriva del activo, el spread y el estilo de salida están dentro de las diez; lo único que cambia entre el filtro y el dado es **cuáles** se quedan.
- **Aísla una sola cosa:** si la condición sabe *elegir*. Una condición que sólo reduce la muestra, sin elegir mejor, da p cerca de 0,5.

Lo que hace el código (`measure.py:52-60`): 10.000 recortes, semilla fija 0, y

```
p = (1 + nº de recortes con estadístico ≥ el de la madre) / (10.000 + 1)
```

El «1 +» evita un p de cero exacto: el menor p posible es 1/10.001 ≈ 0,0001. La prueba es **de una sola cola**: sólo pregunta si la madre es mejor.

Supone que las operaciones de la bolsa son intercambiables: que da igual cuáles se quitan mientras sea el mismo número. Una condición real las quita por rachas. En la corrida de ejemplo, sorteando rachas de 20 operaciones seguidas en vez de operaciones sueltas, la anchura del nulo cambia entre −19 % y +17 % y los falsos «aporta» al 5 % nominal quedan entre el 2 % y el 8 % (calculado por el revisor, sobre las operaciones de `build` y `oos1` del lote `20261006-201740`; con rachas de 50 hay pocas combinaciones para leerse): el supuesto aguanta en USDJPY M30. No está comprobado en otros activos (USA500 tiene 95-129 operaciones en `oos1`) ni con rachas definidas por volatilidad. En las operaciones de las madres de los cinco lotes, `oos1` va de 95 a 737 (mediana 341; calculado por el revisor).

2.4 Las palabras que salen de ahí

`reading.py:31-51`, en este orden:

palabra	cuándo	qué quiere decir
no aplicada	SQX devolvió un fichero que ya no lleva la edición	esa fila no se lee
redundante	la ablación opera exactamente las mismas operaciones que la madre (mismas horas, mismo resultado) y SQX sí corrió el fichero sin el bloque	en esta historia, cuando se cumplen las demás condiciones esta se cumple siempre
no es un filtro	la ablación opera igual o menos que la madre	no recortaba entradas (rama de un OR); el recorte al azar no se puede hacer
resta	ΔE < 0	sin la condición la esperanza por operación es mayor
aporta	ΔE ≥ 0 y p < 0,05	elige mejor que el dado
no se distingue	ΔE ≥ 0 y p ≥ 0,05	opera menos, pero lo que queda no es mejor que un recorte a ciegas

Redundante, con cuidado. No significa «inútil». Significa que en *estos* años nunca se dio el caso en que las demás condiciones se cumplían y esta no. El ejemplo del manual: `Strategy 23.1.53` exige «cierre por encima de la EMA(50)» y «el QQE cruza 57,72»; cuando el QQE cruza ese nivel el precio ya está siempre sobre la EMA. Quitar la EMA devuelve las mismas 694 operaciones. En otra historia la EMA podría filtrar algo.

2.5 Cuándo el recorte al azar es exacto y cuándo no (calculado por mí)

La prueba del apartado 2.3 es exacta si se cumple una cosa: que las operaciones de la madre sean **un subconjunto** de las de la ablación. Entonces la madre es, literalmente, uno de los recortes posibles, y «¿cuántos recortes la igualan?» es una cuenta limpia. Es lo que supone el motor original (`filter.benchmark`, `filter.py:45-48`).

Aquí no se cumple (§2.1, punto 3). El estudio lo sabe y lo resuelve así (`measure.py:33-38`): saca los recortes de la ablación, pero como valor observado usa **la lista propia de la madre**, aunque sus operaciones no estén todas en la ablación.

Eso tiene una consecuencia. Cuando se sacan n operaciones **sin repetir** de una bolsa de N, la media varía menos que si se sacaran de una fuente inagotable: cuanto más se acerca n a N, menos margen hay (con n = N la media es siempre la misma). El nulo se estrecha por un factor √(1 − n/N). Si la madre fuera un subconjunto, ella estaría sujeta a la misma estrechez y todo cuadraría. Pero si sus operaciones son *otras*, su media varía con toda la anchura, y se la compara contra un nulo estrecho: cae en la cola más veces de las que debería.

Lo he medido con las bolsas reales (`mine.py`, 400 repeticiones × 500 recortes por caso): fabrico una «madre» falsa sacando n operaciones **con reposición** de la propia ablación —es decir, sin ninguna ventaja por construcción, e independiente de los recortes— y cuento cuántas veces sale «aporta» a α = 0,05.

caso real	**n / N**	**falsos «aporta» medidos**	**lo que da la fórmula √(1 − n/N) (recalculada a mano; `mine.py` imprime otra expresión, √((1−f)/(1+f)), que da 8,5-25,1 %)**
quitar `BBWRPercent` / `Wasserstein` (quita muchas)	0,18 a 0,25	4,8 % a 8,8 %	6,8 % a 7,7 %
quitar el doble ROC (quita pocas)	0,54 a 0,71	**15,5 % a 20,2 %**	13 % a 19 %
control: madre = subconjunto de verdad	0,69	6,0 %	5 %

(Error de cada porcentaje medido: unos ±2 puntos.)

Cómo leerlo: la tasa real de falsos «aporta» está **entre el 5 % (si la madre fuera subconjunto) y la cifra de la tabla (si no compartiera nada)**. La realidad queda en medio, porque las operaciones no gemelas de madre y ablación se solapan en el tiempo y se parecen. No siempre se compensa: C sin el doble ROC quita pocas operaciones (n/N = 0,67-0,69) y sólo comparte el 31-32 % de las entradas; 7 de las 29 filas de `oos1` están en ese caso (calculado por el revisor: filas con n/N > 0,5 y gemelas < 60 %). No está medido cuánto queda. La cifra de la tabla es la tasa si la madre no compartiera nada con la ablación; no es una cota demostrada (la madre llega a tener 1,2 veces la desviación de su bolsa —A en `build`: 1.002 frente a 838—, y eso sube la tasa; calculado por el revisor). Lo seguro: **un p de 0,03 en una ablación que quita el 30 % de las operaciones no vale lo que dice**. Aplicando esa tasa fila a fila a las 29 de `oos1`, los falsos «aporta» esperados no son 1,45 sino hasta 3,6 (calculado por el revisor).

2.6 La inversión: qué es y qué demuestra

La **inversión** es la misma estrategia entrando y saliendo en los mismos instantes, pero vendiendo donde la madre compra. Pregunta: ¿lo que gana la madre viene de **hacia dónde** entra?

Juguete, una operación. El precio medio (la mitad entre compra y venta) va de 100,00 a 100,50. El spread es 0,04.

```
madre (larga):     compra a 100,02, vende a 100,48      →  +0,46
invertida (corta): vende a  99,98, recompra a 100,52    →  −0,54
```

De esas dos cifras se recuperan dos cosas sin conocer el spread de antemano (`measure.py:126`):

```
movimiento a precio medio = (madre − invertida) / 2 = (0,46 + 0,54) / 2 = 0,50
spread pagado por lado    = −(madre + invertida) / 2 = −(0,46 − 0,54) / 2 = 0,04
```

Es decir: **a precio medio, la invertida gana exactamente lo que la madre pierde y viceversa**. No es un resultado: es una identidad. Si las operaciones están emparejadas al 100 % —y sin stops lo están siempre—, «el invertido pierde lo que el original gana» se cumple en cualquier estrategia, buena o mala.

Entonces, ¿qué informa? El **desglose** del neto en tres trozos:

```
neto de la madre     = +(a precio medio) − spread − (swap y comisión de la madre)
neto de la invertida = −(a precio medio) − spread − (swap y comisión de la invertida)
```

- **A precio medio:** lo que el mercado se movió a favor durante las operaciones. Es la única parte que depende de la dirección.
- **Spread** (aquí incluye el deslizamiento, todo lo que va dentro del precio de ejecución): lo pagan los dos lados por igual.
- **Swap y comisión:** distinto en cada lado. En USDJPY el corto paga mucho más swap que el largo.

Los tres casos posibles:

a precio medio	neto de la madre	lectura
claramente positivo	positivo	gana por la dirección y le sobra para pagar costes
positivo	cero o negativo	acierta la dirección, pero los costes se lo comen
cero o negativo	lo que sea	no acierta la dirección; si el neto es positivo, lo pone el swap

El estudio reduce esto a dos palabras (`reading.py:54-66`): **«en la dirección»** si el precio medio de la madre es mayor que cero, **«sin filo a precio medio»** si no. Y **«no invertida»** si menos del 99 % de las operaciones vuelven emparejadas (misma hora de entrada y de salida, mismo tamaño, lado contrario).

2.7 Lo que la inversión no separa: la deriva del activo

«Gana a precio medio estando larga» tiene dos explicaciones:

- la estrategia **elige cuándo** estar larga (filo), o

- el activo **subió**, y cualquiera que estuviera largo un rato cualquiera habría ganado (deriva).

La inversión no las distingue: las dos dan precio medio positivo en la madre y negativo en la invertida. Para separarlas hace falta saber cuánto habría ganado un largo que estuviera en el mercado **el mismo tiempo, en momentos al azar**. Eso es exactamente lo que mide el estudio del trader al azar (`studies/readings/monkey/`), y por eso los dos se complementan:

	recorte al azar (paso 23)	**trader al azar (monkey, paso 8 y lecturas)**
qué sortea	**cuáles** de las operaciones de la ablación te quedas	**cuándo** entras, con la misma permanencia y los mismos costes
qué deja fijo	las entradas de la estrategia sin el filtro	el mercado, la dirección, el tiempo en mercado
qué pregunta	¿esta condición elige mejor que un dado?	¿la estrategia entera gana más que entrar a ciegas?
la deriva	está dentro de la bolsa: no cuenta ni a favor ni en contra	es justo lo que descuenta

Una condición puede «aportar» en una estrategia que no bate al mono (elige lo mejor de una bolsa mala), y una estrategia puede batir al mono sin que ninguna de sus condiciones por separado se distinga (el filo está en la combinación).

2.8 Sobre qué tramo se lee, y por qué

Tu doctrina: el azar sólo se lee fuera de muestra. La razón es de selección, y se ve con este paso mejor que con ningún otro.

El generador probó miles de combinaciones de condiciones y se quedó con las que mejor resultado daban **en `build`**. Una condición que en `build` no mejorase la estrategia no habría sobrevivido a la búsqueda. Así que preguntar en `build` «¿esta condición elige mejor que un dado?» es preguntar al examen por la nota del que lo ha copiado: sale que sí casi siempre. No es que la prueba esté mal hecha: es que el dado no tuvo miles de intentos y el generador sí.

Los datos lo enseñan sin adornos (los cinco lotes con ablaciones, 30 condiciones):

	`build`	**`oos1`**
aporta	**27**	**1**
no se distingue	2	23
resta	0	5
redundante	1	1

En `oos1`, 1 «aporta» de 29 condiciones no redundantes, cuando el azar solo ya daría 1,45 como mínimo (hasta 3,6 con el nulo estrecho del §2.5; calculado por el revisor). Pero ese recuento no dice que el resto no aporte. Los datos de `oos1` no son los de 29 dados (calculado por el revisor, sobre los 65 JSON guardados de los diez lotes):

- El ΔE es positivo en **24 de las 29** (5 «resta»); con condiciones que no hicieran nada se esperaba la mitad, y la binomial de 24 o más sobre 29 al 50 % es 0,0003. Por madre (ΔE medio de sus dos condiciones): 12 de 15 positivas, binomial 0,018.

- La mediana de los 29 p es **0,20** (con dados sería 0,50); p < 0,10 en **7 de 29** (esperado 2,9), seis de ellos entre 0,054 y 0,096.
- Las condiciones que dan nombre a la plantilla (doble ROC, `IsLower`, `CBlock_CloseCrossesAboveDCUpper`) tienen ΔE > 0 en `oos1` en 14 de 14; las del hueco aleatorio, 10 de 15.
- El encogimiento medio existe: mediana de ΔE `oos1` / ΔE `build` = 0,33 (suma 0,28). En `Strategy 1.13.354` (`IsLower`, USA500) el ΔE pasa de +118,40 a +110,76 (conserva el 94 %), con p = 0,094 y 129 operaciones: no encogió, no cupo.

Estas cifras no prueban que las condiciones aporten: las 29 no son independientes (15 madres, 4 proyectos, operaciones compartidas) y el nulo es estrecho (§2.5); las dos cosas empujan hacia ΔE > 0 y p bajos. Lo que prueban es que el recuento «1 de 30» no distingue entre «no aportan» y «aportan un tercio de lo que decía `build`».

Dos matices sobre `oos1`:

- **Tampoco está limpio, y el sesgo, si existe, va a favor de la madre.** Sólo el paso 8 (R2: neto de `oos1` > 0, con los costes del paso 8) filtró por `oos1`: en la corrida de ejemplo los pasos 10, 12, 14 y 16 dicen «ninguno: el paso no tiene criterio» (`AlgoData/autopilot/Test_USDJPY_M30_doubleROCLong/20261006-201740/resumen.md`; `criteria.yaml:54, 69-80`, `sample: 3`). Las ablaciones no pasaron por ningún filtro. El sesgo a favor de la madre existe en la medida en que el 8 la eligió, y no está medido; A, con −24.603 en el retest, muestra que puede ser pequeño. Aun así «aporta» sólo sale 1 vez, y el ΔE positivo en 24 de 29 se ve también ahí.
- **La falta de «aporta» no es sólo falta de potencia, pero tampoco prueba que no haya efecto.** Con 240-400 operaciones y una desviación de unos 1.000 USD por operación, el recorte al azar necesita un ΔE de 40 a 126 USD para dar p < 0,05. He calculado (aproximación normal, hecha aparte: la cuenta de potencia no está en `recalc.py`; válida aquí, con curtosis en exceso de 1,3 a 10,9 y 240-402 operaciones, calculado por el revisor; no comprobada con las 95-129 operaciones de USA500) qué probabilidad había de detectar en `oos1` un ΔE del tamaño del visto en `build`: 77 %, 95 %, 55 %, 82 %, 30 % y 65 % para las seis condiciones (el revisor reproduce 75, 94, 55, 81, 29 y 65 %). Si los ΔE de `build` fuesen verdad se esperaban **4 detecciones de 6** y hubo 0: eso descarta que el efecto sea el de `build` entero. No descarta un efecto menor: contra la mitad del ΔE de `build` la potencia es del 27 al 50 % y 0 de 6 sale el 6,5 % de las veces; contra un tercio, del 21 al 34 % y 0 de 6 sale el 15 % (calculado por el revisor; potencias que ya incluyen la inflación del §2.5, así que la real es menor). **El paso es ciego en las dos direcciones: «aporta» en `build` no es evidencia y «no se distingue» en `oos1` tampoco lo es de que no aporte.** El ΔE encogió: de +52…+157 en `build` a −42…+57 en `oos1`.

2.9 Por qué es diagnóstico y no selección

Supón que usaras el paso para elegir: «si la ablación sale mejor, me quedo con la ablación». Qué pasaría, estadísticamente:

1. Elegir el mejor de varios hincha el resultado. Juguete: tres versiones de una estrategia (la madre y dos ablaciones) con la **misma** esperanza verdadera, cero. Cada una, medida sobre unos cientos de operaciones, da un número con su error, digamos ±50 USD. Si te quedas con la que mejor salió, de media te quedas con una que marca **+42 USD** (0,84 veces el error; simulado en `toy.py`). No has mejorado nada: has elegido el

ruido que te favorecía. Con los errores reales de `oos1` (49 a 89 USD por operación) son de 41 a 75 USD por operación de ganancia ficticia, del mismo tamaño que la esperanza entera de C (67,75). Como las versiones comparten operaciones el efecto real es algo menor; el signo, no.

2. Casi siempre habrá alguna ablación «mejor». Con dos ablaciones de la misma calidad verdadera que la madre, la probabilidad de que al menos una marque más que ella es del 67 %. «Resta» en una fila no es una rareza: sin más, es lo esperable en una de cada dos condiciones que no hacen nada.

3. El tramo deja de ser fuera de muestra. En cuanto `oos1` decide qué versión sigue, `oos1` es muestra de construcción de esa versión. Todo lo que el workflow midió en `oos1` sobre la madre (pasos 8 a 16) no vale para la versión nueva, que no los ha pasado.

4. Los intentos se multiplican y nadie los cuenta. Cada madre trae consigo tantas versiones como condiciones tiene, más una. Elegir entre ellas son pruebas adicionales que el ledger —el que lleva la cuenta de cuántas cosas se han probado para corregir por ello— no ve.

Por eso el encargo 12 §5 (lo cita el código; el fichero del encargo ya no está en `docs/encargos/`), el README y el aviso de cada página dicen lo mismo: una ablación que sale mejor **no es una estrategia que adoptar**; es una condición que no ayudaba. Quitarla es decisión tuya, se anota, y la estrategia sin ella vuelve a entrar al workflow como una estrategia nueva.

Lo que el paso **sí** sirve para decidir: qué te crees de la estrategia (si su condición «estrella» no se distingue de un dado fuera de muestra, la historia que te cuentas sobre por qué gana tiene menos apoyo), y qué plantillas merece la pena repetir.

---

3. Lo que calcula el estudio, pieza a pieza

Tres pestañas por madre, cada una con un selector de tramo (`build` primero, `oos1` después). Salidas en `<lote>/estudios/structure/`: `estrategias/<madre>.json` y `.html`, `verdict.csv`, `manifest.json`.

3.1 Pestaña 1 — qué aporta cada condición

Una fila por ablación (`contract.py:27-62`, números de `measure.py:63-86`).

columna	fórmula	fichero:línea
ops madre / ops sin ella	nº de operaciones de cada lista	`measure.py:25`
esperanza madre / sin ella	media del `Profit/Loss` por operación, USD	`filter.py:9`
ΔE	esperanza madre − esperanza sin ella	`measure.py:85`
p ΔE	recorte al azar sobre la esperanza (§2.3)	`measure.py:57-60`
Sharpe madre / sin ella	media / desviación típica (poblacional)	`filter.py:10`
ΔSharpe, p ΔSharpe	lo mismo sobre el Sharpe por operación	`measure.py:85-86`
entradas gemelas	entradas de la madre que la ablación también tomó a la misma hora / ops madre	`measure.py:82`
lectura	la palabra del §2.4; **sólo mira ΔE y p ΔE**, nunca el Sharpe	`reading.py:46-51`

Debajo, una barra de ΔE por condición y, por ablación, el histograma de los 10.000 recortes con la línea de la madre.

Tramo `build`, recalculado:

madre	condición quitada	ops madre → sin ella	esperanza madre → sin ella	ΔE	p ΔE	gemelas	lectura
A	doble ROC	736 → 1.140	104,26 → 46,37	+57,89	0,0040	75 %	aporta
A	`BBWRPercent`	736 → 4.144	104,26 → −27,77	+132,03	0,0001	24 %	aporta
B	doble ROC	602 → 1.120	123,85 → 59,12	+64,72	0,0155	54 %	aporta
B	`BBWRPercent`	602 → 2.818	123,85 → −0,70	+124,55	0,0002	36 %	aporta
C	doble ROC	420 → 631	149,48 → 97,57	+51,90	0,0705	31 %	no se distingue
C	`Wasserstein`	420 → 1.883	149,48 → −7,14	+156,62	0,0022	67 %	aporta

Tramo `oos1`, recalculado:

madre	condición quitada	ops madre → sin ella	esperanza madre → sin ella	ΔE	p ΔE	gemelas	lectura
A	doble ROC	402 → 563	−61,20 → −69,96	+8,76	0,362	77 %	no se distingue
A	`BBWRPercent`	402 → 2.130	−61,20 → −18,72	**−42,49**	0,854	20 %	**resta**
B	doble ROC	331 → 555	0,89 → −44,66	+45,55	0,109	57 %	no se distingue
B	`BBWRPercent`	331 → 1.478	0,89 → −5,92	+6,81	0,450	32 %	no se distingue
C	doble ROC	240 → 348	67,75 → 50,42	+17,34	0,352	32 %	no se distingue
C	`Wasserstein`	240 → 974	67,75 → 10,44	+57,31	0,229	63 %	no se distingue

Cómo leer una fila, con A sin `BBWRPercent` en `build`: sin el filtro la estrategia hace 4.144 operaciones y **pierde** 27,77 USD en cada una; con él hace 736 y gana 104,26. De 10.000 recortes al azar de 736 operaciones, ninguno llegó a 104,26 (la media de los recortes fue −27,75, con una desviación de 27,5). p = 1/10.001.

La misma condición, en `oos1`: sin el filtro pierde 18,72 por operación; con él pierde **61,20**. El filtro que en `build` separaba lo bueno de lo malo, fuera de muestra elige peor que no filtrar. La lectura es «resta».

Sobre ese «resta». La palabra sale por ΔE < 0, sin prueba. He calculado la cola contraria (qué fracción de los recortes sale igual o **peor** que la madre): 0,147. Uno de cada siete recortes a ciegas habría salido tan mal. No es evidencia de que el filtro estropee; es evidencia de que no ayuda.

3.2 Pestaña 2 — la misma entrada, al revés

Una tabla por tramo (`contract.py:65-85` , números de `measure.py:89-132`).

fila	fórmula	línea
operaciones	las de cada fichero	`:122`
esperanza neta por operación	media del `Profit/Loss`	`:131-132`
neto	suma del `Profit/Loss`	`:129-130`
bruto a precio de ejecución	Σ lado × (cierre − apertura) × tamaño × valor del punto	`:115-119, 125`
a precio medio	(bruto madre − bruto invertida) / 2; el de la invertida es su negativo	`:126`
swap y comisión	bruto − `Profit/Loss` , lo que SQX cobró de verdad	`core/trades.py:43`
emparejadas (pie de tabla)	fracción con misma entrada, misma salida, mismo tamaño y lado contrario	`:110-113, 123`
correlación del bruto (pie)	entre el bruto por operación de las dos	`:124`
spread por lado (pie)	−(bruto madre + bruto invertida) / 2	`:126`

Recalculado (USD):

tramo	madre	a precio medio	spread	swap+com. madre	swap+com. invertida	neto madre	neto invertida	lectura
`build`	A	+110.283	18.514	15.034	27.136	+76.735	−155.933	en la dirección
`build`	B	+101.775	15.334	11.884	27.365	+74.556	−144.474	en la dirección
`build`	C	+76.856	8.973	5.103	31.969	+62.780	−117.798	en la dirección
`oos1`	A	**−56**	14.396	10.151	22.174	−24.603	−36.513	sin filo a precio medio
`oos1`	B	+19.696	11.690	7.712	22.584	+294	−53.970	en la dirección
`oos1`	C	+26.605	6.957	3.387	23.805	+16.261	−57.367	en la dirección

Emparejadas: 100 % en las seis. Correlación del bruto: −0,9995 a −0,9999.

La cuenta cuadra al céntimo. A en `build`: 110.283 − 18.514 − 15.034 = 76.735; y −110.283 − 18.514 − 27.136 = −155.933.

Tres lecturas que la tabla da y la palabra no:

- **A en `oos1`:** a precio medio, cero (−56 USD en 402 operaciones). La madre pierde 24.603 y **la invertida también pierde**, 36.513. Las dos pierden lo mismo por la misma razón: spread y swap. El manual dice de «sin filo» que «invertida gana lo que ella pierde»; eso es cierto a precio medio (+56) y falso en neto.
- **B en `oos1`:** «en la dirección» con un neto de +294. Acierta 19.696 a precio medio y paga 19.402 de costes. La palabra es verde; la estrategia no gana.
- **El invertido pierde mucho más de lo que la madre gana**, siempre, y la diferencia no dice nada de la estrategia: es dos veces el spread más el swap y la comisión de las dos patas (el corto de USDJPY paga de 1,8 a 7 veces más; cociente de las seis filas de la tabla). Comprobación con A en `build`: 155.933 − 76.735 = 79.198 = 2 · 18.514 + 15.034 + 27.136.
- **Cuánto de lo que pierde el invertido es coste y no dirección** (calculado por el revisor, con las cifras de la tabla): 29 %, 30 % y 35 % en `build`; en `oos1`, 100 % (A), 64 % (B) y 54 % (C). Descontada además la deriva del §3.3, la parte de dirección que queda es el 21 % (B) y el 32 % (C) de lo que pierde el invertido.

3.3 Cuánto de «a precio medio» es deriva (calculado por mí)

El estudio no lo calcula. Lo he estimado así (`mine.py`): con las barras M30 del feed, la subida media por barra en el tramo; multiplicada por las barras que cada operación estuvo abierta, por su tamaño y por el valor del punto. Es lo que habría ganado a precio medio un largo con los mismos tamaños y las mismas permanencias puesto en momentos cualesquiera.

tramo	USDJPY (cierre)	madre	tiempo en mercado	a precio medio	de ello, deriva	resto	error típico de la suma
`build`	111,67 → 112,66	A	10,9 %	+110.283	+301	+109.982	±27.236
`build`		B	11,2 %	+101.775	+313	+101.462	±28.159
`build`		C	13,2 %	+76.856	+312	+76.544	±24.140
`oos1`	112,61 → 131,07	A	11,9 %	−56	+7.890	**−7.946**	±19.490
`oos1`		B	12,4 %	+19.696	+8.104	+11.592	±18.532
`oos1`		C	15,4 %	+26.605	+8.327	+18.278	±21.263

En `build` el yen acabó donde empezó y la deriva uniforme es despreciable; calculada año a año son 5.900-9.100 USD, del 5 al 12 % del precio medio, y el resto sigue a 2,8-3,8 errores típicos (calculado por el revisor, sólo en USDJPY M30). En `oos1` subió un 16 %, y un largo cualquiera con esas permanencias se habría llevado unos 8.000 USD: el 41 % del precio medio de B y el 31 % del de C (con deriva año a año, 5.758 / 6.937 / 8.639 en vez de 7.890 / 8.104 / 8.327: mismo orden; calculado por el revisor). A, descontada la deriva, queda en −7.946 ±19.490: no se distingue de estar largo a ciegas (0,4 errores típicos).

La última columna es lo que varía la suma por el mero ruido de las operaciones (desviación por operación × √n). En `oos1` ninguna de las tres tiene un precio medio que se separe de cero ni a dos errores: 0,0, 1,1 y 1,3. La etiqueta «en la dirección» de B y C descansa sobre eso.

Supuestos de la estimación: deriva repartida por igual entre todas las barras del tramo (no lo está: una estrategia que entra tras subidas puede coger tramos con más deriva que la media); tamaños como los operó SQX. Es una orden de magnitud, no una medida. La medida de verdad es el trader al azar.

3.4 Pestaña 3 — controles

Tres comprobaciones por tramo (`contract.py:88-116`). El veredicto de la pestaña es «pass» sólo con las tres en verde; si no, «watch».

control	cómo	corrida real
SQX conservó cada edición	`retained.parquet` : cada fichero reteseado se relee y se compara con el plan (`keep.py:44`)	pass (12 de 12 ficheros en cada pata)
alguna ablación cambia el nº de operaciones	cuenta las ablaciones con distinto nº de operaciones que la madre (`contract.py:92`)	pass (2 de 2 en cada madre)
la madre reconstruida reproduce un resultado guardado	nº de operaciones igual y neto dentro del 0,5 % de lo que el `.sqx` de la madre guarda para ese tramo (`measure.py:150-158`)	**«none» en los seis casos**

El tercero, con datos:

madre	tramo	guardado en el fichero (código «IS»)	reconstruida	igual
A	`build`	402 ops / −24.602,68	736 / +76.734,87	False
B	`build`	331 / +293,66	602 / +74.556,08	False
C	`build`	240 / +16.261,16	420 / +62.780,39	False
A, B, C	`oos1`	no hay muestra con código «OOS»	—	sin fila

Lo «guardado como IS» es, al céntimo, el resultado de `oos1` de la propia reconstrucción (A: 402 / −24.602,68; B: 331 / +293,66; C: 240 / +16.261,16). Lo guarda porque la madre sale de la databank `SPP OOS` (`pipeline/autopilot/criteria.yaml:226`, `pipeline/autopilot/survivors.py:30`), cuya tarea corre sobre `oos1` (`assets/_build.yaml:403`); un retest escribe su resultado en el bloque «IS» y deja el 20 a cero (`studies/screening/gate/collect.py:50-55`; comprobado en las tres madres, bloque 10 = 402 / −24.602,68 en A). Comprobado en los diez lotes (calculado por el revisor, sobre los JSON guardados): en 59 de 59 madres con «igual: False», lo guardado como «IS» es la reconstrucción en `oos1`; las tres madres del lote de 2026-09-27, que no vino del autopiloto, dan «igual: True». En cualquier caso: **la reconstrucción reproduce al céntimo el resultado guardado de `oos1`**, pero el estudio la compara con el tramo equivocado y no lo ve; la de `build` no se puede contrastar con este fichero, que no guarda nada de `build`. Arreglar la casilla (leer el bloque que el fichero rellena, como hace `collect.py`) sólo puede poner en verde el control de `oos1`; el de `build` seguirá en «none», y como el veredicto de la pestaña es por tramo (`contract.py:88-116`), la de `build` seguirá en «watch». Gravedad: que la versión ablacionada sea «la madre menos una condición» no lo garantiza este control sino las dos relecturas del fichero (`make.py:104-110`, `keep.py:65-67`), que sí pasan (12 de 12). Lo que el control añadiría —que la madre reconstruida da el backtest de la madre— hoy no lo comprueba el estudio, pero se cumple en los datos de `oos1`.

Con madres sin ablaciones (todas las de NIKKEI225) el segundo control da «fail» por construcción: no hay ninguna ablación que pueda cambiar nada.

3.5 `verdict.csv` y el resumen

Una fila por madre: `strategy`, `identity`, `verdict` (siempre `info`) y una columna por lectura, `<bloque>@<tramo>` e `inversion@<tramo>` (`one.py:77-79`, `report.py:40`).

```
strategy            AND@build        CBlock_BBWRPercent@build  AND@oos1
CBlock_BBWRPercent@oos1  inversion@build  inversion@oos1
Strategy 10.23.380  aporta           aporta                    no se distingue  resta
en la dirección  sin filo a precio medio
```

Las columnas dependen de los bloques de cada madre: en un lote con bloques distintos la tabla sale con huecos.

3.6 Lo que queda en el lote además del estudio

fichero	quién lo escribe	contenido
`structure.parquet`	`make`	el plan y lo que cada fichero contiene de verdad
`sqx/*.sqx`	`make`	los ficheros fabricados
`ran.json`	`execute`	las tres patas, su tramo y su databank
`retest_build.csv`, `retest_oos1.csv`, **`retest_oos2.csv`**	`execute`	el panel de métricas de SQX de cada pata
`retested/WFC_Build`, `WFC_OOS1`, **`WFC_OOS2`**	`keep`	los `.sqx` que devolvió SQX
`retained.parquet`	`keep`	si cada fichero conserva su edición, **en las tres patas**

---

4. El criterio cargado y por qué

4.1 Tal como está hoy

- **`pipeline/autopilot/criteria.yaml`**: el paso 23 **no tiene entrada**. Línea 59: «Steps 21, 23, 24 annotate only: no rule on purpose».
- **`ledger/thresholds.yaml`**: ninguna línea del estudio.
- **`pipeline/autopilot/facts.py`**: no hay extractor propio. Si algún día se le diera una regla, los hechos saldrían del aplanado genérico de los JSON (§4.3).
- **Autopiloto**: `tail.py:16` lo trata como un «lote» (`BATCH = {"23": "structural", …}`). Lo ejecuta para **todas** las supervivientes, no lo juzga (sólo se juzga un paso con reglas, `tail.py:44`) y no escribe `paso-23/hechos.parquet`. En la corrida real no existe esa carpeta.
- **`survivors.py:14`**: el 23 está en la lista de pasos que *podrían* declarar un corte «el día que el dueño le dé reglas». Hoy no lo declara.

4.2 Los cinco mandos del estudio

`studies/readings/structure/config.yaml`. Ninguno quita una estrategia.

mando	valor	qué mueve
`subsets.draws`	10.000	resolución del p: el mínimo es 0,0001 y el error de un p de 0,05 es ±0,002
`subsets.seed`	0	fija: dos lecturas del mismo lote dan el mismo p
`verdict.alpha`	0,05	la frontera entre «aporta» y «no se distingue»
`controls.identity_tolerance`	0,005	diferencia relativa de neto que aún es «el mismo backtest»
`controls.min_paired`	0,99	por debajo, la inversión es «no invertida»

Si mueves α. Sobre las 6 condiciones de la corrida en `oos1`: con 0,05, 0 «aporta»; con 0,10, 0; con 0,15, 1 (B sin el doble ROC, p = 0,109). En `build`: con 0,05, 5 de 6; con 0,01, 4 de 6; con 0,10, 6 de 6. Mover α no cambia la foto en esta corrida. En los cinco lotes sí: de 1 «aporta» con α = 0,05 a 7 con α = 0,10 (p = 0,003,

0,054, 0,057, 0,069, 0,076, 0,094, 0,096); en la franja 0,05-0,10 se esperaban 1,45 y hay 6 (calculado por el revisor). El «1 de 30» depende del umbral.

4.3 Qué hechos saldrían si se le pusiera una regla

He pasado el JSON de A por `facts.flatten` (`saved.py`): **68 hechos numéricos**. Tres problemas para escribir una regla con ellos:

1. **Sólo el tramo por defecto del selector**, que es el primero: `build` (`facts.py:71-73`; el orden lo da `closing.py:89`). De `oos1` no saldría ningún hecho.
2. **El nombre del bloque va dentro de la clave**: `conditions.m_por_condicion.cblock_bbwrpercent.p_e` (el prefijo `structure.` lo añadiría quien llama a `facts.flatten`). Cada estrategia tiene claves distintas; una regla necesita un nombre fijo.
3. **Las etiquetas son texto** y `facts` descarta lo que no es número. `conditions.verdict_pass` sale repetido, una vez por condición, y se numera `#2`.

4.4 Las comprobaciones que sí paran la cadena

No juzgan estrategias; protegen de leer un lote roto.

comprobación	dónde	efecto
el fichero fabricado no es el plan	`make.py:109-110`	`SystemExit`, no se corre nada
madre con stop/target/trailing	`logic.py:153`	`ValueError` en `make`: cae el lote
el custodio sigue arriba y no es de esta sesión	`keep.py:59-60`	`SystemExit`
SQX no conservó una edición	`keep.py:65-67`	`SystemExit`
ningún export lleva las variantes del lote	`inputs.py:63-65`	`SystemExit`
`--databank WFC_OOS2` como agente autónomo	`ledger/gate.py:73-79`	`PermissionError`

---

5. Lo que no cuadra o no está medido

5.1 Las estrategias sin plantilla no se ablacionan, y nadie lo dice

`sqx/structural/logic.py:10`: `TOP = re.compile(r'\s*<Item key="(AND|OR)">')`. Exige el elemento exactamente así. Y `logic.py:11, 47-59` busca los hijos como `<Block>`.

Una señal escrita por una plantilla tiene esa forma:

```
<signal …><Item key="AND"><Block><Item key="AND">…</Item></Block><Block><Item
key="CBlock_BBWRPercent">…
```

Una señal cuyo AND sorteó el generador, no:

```
<signal …><Item key="AND" generated="random" randomId="RandomConditionLong" gid="…"><Item …
key="KCBarOpensAboveUpperAfterOpenBelow" …>…<Item … key="CBlock_ZScoreStretchedUp" …>
```

`TOP` no casa, `conditions()` devuelve una única fila con `block = "AND"` y `operator = ""`, y `plan.py:46` la salta. Recuento (`unablated.py`):

proyecto	madres	con AND de 2+ condiciones	con ablaciones fabricadas
`Trade_NIKKEI225_M30_volatilidadL`	10	9	**0**
`Test_NIKKEI225_M30_simpleL_auto`	10	4	**0**
`Test_NIKKEI225_M30_simpleL_fin`	10	3	**0**
`Test_NIKKEI225_M30_simpleL_live`	10	2	**0**
`Test_NIKKEI225_M30_simpleL_ref`	10	0	0
`Test_USA500_H1_indicesIBS`	3	3	3
`Test_USDJPY_M30_doubleROCLong`	3	3	3
`Test_USDJPY_donchianUpperCrossUp_H1`	3	3	3

El paso se da por «hecho», la pestaña 1 sale vacía y el control 2 da «fail». El manual (`51-estructura.md`, «Si algo falla») lo explica como «la señal de entrada tiene una sola condición». El test de la fábrica (`tests/test_structural.py`) sólo tiene dos ficheros de ejemplo, los dos de plantilla.

Alcance de los recuentos (calculado por el revisor): las 30 condiciones salen de 15 madres de 4 proyectos, todos de plantilla, 2 activos (12 madres de USDJPY, 3 de USA500); de Simple Strategy hay cero condiciones medidas. En cada madre una de las dos condiciones es la fija de la plantilla, que no eligió el generador sino quien escribió la plantilla. La inversión sí cuenta esas madres sin ablación: 50 de las 65 son de NIKKEI225, así que los recuentos «65 de 65» y «44 de 65» son en tres cuartas partes de un solo activo.

5.2 El control de identidad compara contra el tramo equivocado

§3.4. `measure.py:12` asume que el código de muestra 10 del fichero es `build` y el 20 es `oos1`. En el fichero que llega por el autopiloto el 10 es `oos1` y el 20 no existe. Resultado en el autopiloto: «none» siempre, veredicto de controles «watch» siempre (en el lote de 2026-09-27, que no vino del autopiloto, las 3 madres dan «igual: True»; calculado por el revisor). El manual (`51-estructura.md`, pestaña 3) dice «sin los tres en verde no hay lectura»; con esa regla no habría lectura nunca. `OPEN.md:582` anota un caso emparentado (tabla vacía con madres de `CrossTF_Mothers`); en esta corrida no está vacía: está llena con una comparación que no procede, y ese caso no está anotado. La causa: la madre sale de la databank `SPP OOS`, que corre sobre `oos1` (§3.4); el fichero no guarda nada de `build`, así que arreglar la casilla no daría el control de `build`. La gravedad es acotada: las dos relecturas del fichero (`make.py:104-110`, `keep.py:65-67`) sí pasan.

5.3 El recorte al azar no es exacto con listas no anidadas

§2.5. El README (`studies/readings/structure/README.md:26-32`) lo plantea como un detalle y cita un 2 % de gemelas en una estrategia. En esta corrida van del 20 % al 77 %, y el efecto sobre el p no está medido: sólo la tasa si la madre no compartiera nada con la ablación (falsos «aporta» de hasta el 15-20 % cuando se quitan pocas operaciones; no es una cota demostrada).

5.4 Dos de las palabras no llevan prueba

- **«Resta»** (`reading.py:49-50`): ΔE < 0, sin más. Asimétrico con «aporta», que exige p < 0,05. De los 5 «resta» en `oos1` de los lotes en disco, el único que he podido recalcular tiene una cola contraria de 0,147.
- **«En la dirección»** (`reading.py:66`): precio medio > 0, sin error ni deriva. En `oos1` las dos que lo reciben están a 1,1 y 1,3 errores típicos de cero (§3.3).

Y en las 65 madres de los diez lotes: «en la dirección» en `build` , **65 de 65** (en `oos1` , 44 de 65, y sólo 26 de 65 con neto positivo; calculado por el revisor). Una palabra que sale siempre no informa: una estrategia larga que no ganase a precio medio en `build` no habría salido del generador.

5.5 La deriva no se separa

§2.7 y §3.3. Importa sobre todo en índices: en los cinco lotes de NIKKEI225 (50 madres largas, sin ablaciones) la inversión es lo único que el paso produce, y dice «en la dirección» en `build` 50 de 50 y en `oos1` 30 de 50. Sin descontar la subida del índice no se sabe cuántas de esas 30 lo son.

5.6 El paso lee `build` por defecto

§2.8. `WORKFLOW.md:53` y `:145` declaran que el paso lee `build` y `oos1` ; es coherente con el código. Lo que no cuadra con la doctrina «el azar sólo en OOS» es que el p del recorte al azar se calcule y se pinte en `build` con la misma etiqueta verde, y que sea el tramo que la ventana abre primero y el único que llegaría a los hechos.

5.7 `oos2` se reprueba y queda en disco

`closing.py:55` llama a `sqx.variants.execute` , que corre las tres patas (`ran.json` : `WFC 3 OOS2` , 12 en disco). La puerta sólo se pregunta por `build` y `oos1` (`closing.py:85-86`). Quedan `retest_oos2.csv` , `retested/WFC_OOS2/` y las filas `oos2` de `retained.parquet` . El manual lo avisa («No lo abras»). No es una lectura del estudio, pero es un backtest de versiones nuevas de la estrategia sobre el tramo reservado, sin fila en el ledger.

Además, el README (`:35-36`) dice que `inputs.load` «asks the ledger's one-way door for every leg and records the look». `ledger/gate.py:68-69` : `allow` devuelve sin hacer nada si no es un agente autónomo, y cuando lo es sólo acepta o rechaza. **No apunta nada.** La misma afirmación falsa está en tres sitios más: el docstring de `allow` (`ledger/gate.py:59-61` : «the row is still written»; el código no escribe la fila nunca), `assets/_policy.yaml:44-45` («el ledger sigue contando cada mirada») y el README `:35-36` . Y `core/assetdata.py:25` dice «Nothing sets it today»: nadie pone `ALGO_AUTONOMOUS` , así que el `PermissionError` de `gate.py:73-79` es hoy inalcanzable y el estudio no se niega a leer `oos2` nunca.

5.8 Dos lotes del mismo proyecto el mismo día se pisan

Los identificadores son genéricos (`S00000` , `S00A01` …, `plan.py:42`) y el export va a `raw/<proyecto>/<databank>/<día>/structure/trades.parquet` . El 2026-10-06 este proyecto tuvo dos lotes (`…-163339` , madres `14.22.275` , `8.16.266` , `9.23.278` ; y `…-201740` , las de este documento). El segundo export, a las 21:37, **sobrescribió** el del primero. Los JSON del primer lote son correctos (se

escribieron antes). Pero si se relanzara su `report`, `inputs.latest` aceptaría el export nuevo —lleva los mismos doce identificadores— y atribuiría a `14.22.275` las operaciones de `10.23.380`, sin error. El README (`:39-47`) dice que `latest` «reads the batch's OWN export».

5.9 Cosas menores

- **«AND» como nombre de condición.** Un grupo anidado (el doble ROC) aparece como «AND» en la tabla, en la barra y en la columna del CSV (`logic.py:85`).
- **Dos bloques iguales en la misma señal** comparten clave en el resumen y la segunda pisa a la primera (`one.py:77`).
- **Ocho p por madre** (2 condiciones × 2 tramos × 2 estadísticos) sin corrección; la etiqueta sólo usa la esperanza.
- **La esperanza está en USD con tamaño variable.** En A las operaciones van de 0,98 a 16,76 lotes (`build` 0,98-16,15; `oos1` 1,28-16,76): la media por operación pesa más las de más lotes. Afecta igual a madre y ablación, pero no es «pips por operación».
- **Memoria del sorteo.** `measure.py:57` construye una matriz de 10.000 × N: con N = 4.144 son unos 330 MB por matriz; con una ablación de 20.000 operaciones, 1,6 GB. Coexisten dos matrices (`np.tile` y `rng.permuted`, `measure.py:57`), así que el pico es el doble: ≈ 660 MB con N = 4.144 (medido por el revisor con tracemalloc, 10.000 recortes) y ≈ 3,2 GB con 20.000.
- **El manual está desfasado en el ejemplo**: proyecto sin prefijo `Test_`, feed `USDJPY_DukasM1_the5ers`, costes «provisionales» de septiembre, «cuatro ficheros por tres patas y diez mercados».
- **Una madre con stop tumba el lote entero** (§1.4), no sólo su inversión.

5.10 Lo que no está medido

- La tasa real de falsos «aporta» con el solape que hay (sólo la tasa si no compartieran nada, que no es una cota).
- Cuánto hincha la esperanza de la madre en `oos1` el haber sido elegida por el paso 8 con su `oos1` (los pasos 10-16 no filtran).
- **Interacciones**: se quita una condición cada vez. Dos condiciones que se tapan salen las dos como poco útiles.
- **Las salidas**: sólo se ablacionan entradas. Las dos madres de NIKKEI225 que he abierto tienen señales de salida con dos condiciones.
- **Ninguna población sin ventaja** ha pasado por el paso: no se sabe qué reparto de palabras da el ruido.
- No hay ningún test de `measure.py` ni de `reading.py`; los tests cubren la fábrica y el orden de llamadas del autopiloto.

5.11 El «1 de 30» en `oos1` no separa «no aporta» de «aporta menos»

Descrito en el §2.8. Lo que hay: ΔE positivo en 24 de 29 condiciones no redundantes, mediana de p 0,20, 7 de 29 con p < 0,10, y una potencia del 21 al 50 % contra un efecto de un tercio o de la mitad del visto en `build` (todo calculado por el revisor; potencia con aproximación normal, no comprobada en USA500, con 95-129 operaciones en `oos1`). Con las muestras que hay, el paso no distingue «no aporta» de «aporta una parte de lo visto en `build`»: es ciego en las dos direcciones.

5.12 Por qué la madre guarda `oos1` como «IS»

Lo que hay: la madre que copia el autopiloto sale de la databank `SPP OOS` (`criteria.yaml:226`, `survivors.py:30`), cuya tarea corre sobre `oos1` (`_build.yaml:403`), y un retest rellena el bloque «IS» y deja el 20 a cero (`collect.py:50-55`). El fichero no guarda ningún resultado de `build`. Leer en la casilla que el fichero rellena daría el control del tramo `oos1`, no el de `build`, que seguiría en «none» (§3.4).

5.13 Las versiones reprobadas en `oos2` sin fila en el ledger

Lo que hay (calculado por el revisor; sólo existencia de ficheros, contenido sin abrir): en `AlgoData/structural/` hay 11 ejecuciones, las 11 con `retest_oos2.csv` y con `oos2` en `ran.json`. Son 180 ficheros fabricados, de ellos 105 versiones que no son la madre (30 ablaciones y 75 inversiones), cada una reprobada en `oos2`. En `AlgoData/ledger/*.jsonl` hay 0 filas del paso 23 (del 24 hay 73, del 22 hay 3). La tabla del cierre (§6.1) ponía «coste en estrategias: 0» sin fila para este gasto de datos reservados.

5.14 La diferencia en la inversión, y «peor que estar largo a ciegas»

Lo que hay: la diferencia entre lo que pierde el invertido y lo que gana la madre es 2·spread más los swaps y comisiones de las dos patas, no la diferencia de swap (§3.2). Y en A, en `oos1`, descontada la deriva, el precio medio queda en −7.946 con un error típico de ±19.490 (t = −0,41): no se distingue de cero, así que «peor que estar largo a ciegas» no está demostrado (§3.3).

---

6. Cierre

6.1 El paso, en una tabla

	hoy
reglas en `criteria.yaml`	ninguna, a propósito
umbrales en el ledger	ninguno
qué decide	palabras por condición y por tramo; ningún superviviente
tramos	`build` y `oos1`; `oos2` se reprueba pero no se lee
coste	143 s para 3 madres
coste en estrategias	0: todas siguen al 24
coste en datos reservados	cada lote reprueba todas sus versiones en `oos2` y deja el panel en claro (`retest_oos2.csv`). Hasta hoy: 11 ejecuciones, 105 versiones nuevas (30 ablaciones, 75 inversiones), 0 filas del paso 23 en el ledger (calculado por el revisor). Nadie puede demostrar después que no se miraron

6.2 El resultado sobre lo que hay en disco

	`build`	`oos1`
condiciones que «aportan» (30, cinco lotes)	27	1
inversión «en la dirección» (65 madres, diez lotes)	65	44
madres con dos condiciones y sin ablación	18 de 50 (NIKKEI225)	
control de identidad en verde (diez lotes, 62 madres con fila en `build`)	3 de 62 (las 59 restantes coinciden con `oos1`, el tramo que el control no mira; calculado por el revisor)	sin fila

Dicho llano: **fuera de muestra, 1 de 29 condiciones llega a p < 0,05, que es lo que da el azar; pero el paso tampoco demuestra que no aporten.** En `oos1` el ΔE es positivo en 24 de 29 (con condiciones sin efecto se esperaba la mitad), la mediana de p es 0,20 y 7 de 29 quedan por debajo de 0,10 (calculado por el revisor). Parte de ese sesgo lo ponen la elección de la madre por el paso 8 y el nulo estrecho del §2.5, y no está medido cuánto. Con 95-737 operaciones por madre, la etiqueta no separa «no aporta» de «aporta un tercio de lo visto en `build`». Lo que el paso no hace bien es (a) funcionar con las estrategias sin plantilla, (b) comprobar que la madre reconstruida es la madre, y (c) distinguir filo de deriva en la inversión.

6.3 Las decisiones que te quedan

Ninguna cuesta estrategias mientras el paso siga siendo diagnóstico (sí cuesta datos reservados: ver la tabla 6.1 y §5.13).

#	decisión	qué ganas	qué cuesta
1	**¿Se arregla que la fábrica lea las señales del generador (5.1)?**	el paso pasa de «sólo inversión» a completo en las corridas sin plantilla, que son las que más corres	un cambio en `logic.py` y un fichero de ejemplo más en su test; hasta entonces, la pestaña 1 vacía no significa «una sola condición»
2	**¿El control de identidad compara contra el tramo que el fichero guarda de verdad (5.2)?**	un control que puede ponerse en verde; hoy la pestaña 3 es siempre «watch» y se aprende a ignorarla	decidir de dónde se copia la madre o contra qué muestra se compara; ojo: la madre sale de `SPP OOS` y su fichero no guarda nada de `build`, así que arreglar la casilla sólo pone en verde el control de `oos1`; el de `build` seguiría en «none» y su pestaña en «watch» (§5.12)
3	**¿En qué tramo se lee el recorte al azar?** Sólo `oos1`, o `build` como referencia sin etiqueta verde	coherencia con tu doctrina; el 27/30 de `build` deja de parecer un resultado	perder la comparación `build` → `oos1`, que es la que enseña cuánto encoge cada condición
4	**¿«Resta» y «en la dirección» llevan prueba, o cambian de nombre?**	palabras que no afirman más de lo que hay	hace falta decidir contra qué se prueba la dirección: contra cero, o contra la deriva (el mono)
5	**¿La inversión descuenta la deriva?**	en índices, saber cuántos «en la dirección» son la subida del índice	es la misma pregunta que ya contesta el trader al azar: quizá baste enlazar su resultado en la pestaña 2 en vez de calcular nada nuevo
6	**¿El lote del paso 23 se reprueba en `oos2`?**	que el tramo reservado no reciba backtests de versiones que nadie va a leer	`execute` corre hoy las tres patas para todos los lotes; habría que poder decirle cuáles
7	**¿Va a tener el paso alguna regla?** Hoy no puede: los hechos no tienen clave fija ni tramo `oos1` (4.3)	—	antes de escribirla haría falta una población sin ventaja que diga qué reparto de palabras da el ruido
8	**¿Qué haces con una condición que «no se distingue» o «resta» en `oos1`?**	—	quitarla es fabricar una estrategia nueva: vuelve al principio del workflow y sus pruebas cuentan en el ledger (§2.9). No hay atajo

6.4 El límite que no cambia

El paso mira estrategias que ya han sido elegidas, sobre tramos que ya se han usado para elegirlas: cierto para `build` (el generador) y, en parte, para `oos1` (sólo el paso 8 filtró por él). Puede decirte que una condición **no** se sostiene fuera de muestra, y eso es información. No puede decirte que una versión distinta sea mejor: en cuanto lo usas para eso, deja de medir y empieza a buscar.